

Web Engineering 3

Vorlesung 6

Hochschule Zittau/Görlitz

Christopher-Manuel Hilgner

Agenda

Vorlesung

- Spring Testing
- Unit Testing
- Integration Testing

Seminar

- Tests Schreiben

Unit-Tests

Unit-Tests

- Tests von einzelnen Komponenten

Was machen gute Unit Tests aus?

- sind isoliert: Sie sind voneinander unabhängig, sodass die Reihenfolge ihrer Ausführung das Testergebnis nicht beeinflusst. Schlägt ein Test fehl, so führt dies nicht dazu, dass weitere Tests fehlschlagen.
- sichern jeweils genau eine Eigenschaft ab. Ein Problem äußert sich immer in genau einem fehlschlagenden Test.
- sind vollständig automatisiert, damit sie auch bei erhöhtem Projektdruck noch häufig ausgeführt werden.
- sind leicht verständlich und kurz.

...

Unit Testing in Spring

- Springs Architektur macht isoliertes Testen sehr einfach
- Genutzt werden: *Mock Objects*
- *Mock Objects* ersetzen alle Objekte, die nicht direkt getestet werden sollen
- Ermöglichen zum Beispiel Tests für Service Layer die Repositories benötigen
- Repositories müssen nicht auf persistente Daten zugreifen
- Es wird keine laufende Datenbank benötigt

Unit Testing in Spring

- Unit Tests brauchen nicht zwingend Funktionalitäten, die Spring bereitstellt
- Spring bringt Features mit, die Testing vereinfachen

Vorteile von Unit Tests:

- Schnelle ausführung bei richtiger Konfiguration
- Es wird keine Laufzeit Infrastruktur benötigt

Spring Testing Komponenten

Mocking:

- Environment
- Servlet API
- Spring Web Reactive

Mocking

Mocking

Environment

- `org.springframework.mock.env` Package bringt Implementation für `Environment` und `PropertySource`
- `MockEnvironment` und `MockPropertySource` können genutzt werden
- Genutzt für *out-of-container* Tests die Environment spezifische Werte benötigen

Mocking

Servlet API

- `org.springframework.mock.web` Package enthält Mock Servlet API Objekte
- Testing von Web Context, Controller und Filter
- Nutzung in Kombination mit Spring Web MVC
- Praktischer als dynamische Mock Objekte
- MockMVC als Erweiterung für Integrations-Tests für Spring MVC

Mocking

Spring Web Reactive

- `org.springframework.mock.http.server.reactive` Package enthält mock Implementationen für:
 - `ServerHttpRequest`
 - `ServerHttpResponse`
- Nutzung in WebFlux Anwendungen
- `org.springframework.mock.web.server` Package mit Mock Implementation von `ServerWebExchange`

Mocking

Mockk Beispiel

Mockk Beispiel

```
1  class UserServiceTest {
2      private val userRepository: UserRepository = mockk()
3      private val userService = UserService(userRepository)
4      @Test
5      fun testCreateUser() {
6          val newUserDto = CreateUserDto(
7              username = "New User"
8          )
9          every { userRepository.save(any()) } returns User(
10             username = "New User"
11         )
12         val actual = userService.createUser(newUserDto)
13         assertEquals(newUserDto.username, actual.username)
14     }
15 }
```

◀ Kotlin

Mockk Beispiel

```
1 private val userRepository: UserRepository = mockk()  
2 private val userService = UserService(userRepository)
```

◀ Kotlin

- Repository wird gemocked
- Funktionen im Repository haben keine direkten Funktionen und müssen später definiert werden
- Mock Repository wird an Service übergeben

Mockk Beispiel

```
1  @Test
2  fun testCreateUser() {
3      val newUserDto = CreateUserDto(
4          username = "New User"
5      )
6      every { userRepository.save(any()) } returns User(
7          username = "New User"
8      )
9      val actual = userService.createUser(newUserDto)
10     assertEquals(newUserDto.username, actual.username)
11 }
```

 Kotlin

} (1)

- Zu erstellender User wird mit einem DTO erstellt

Mockk Beispiel

```
1  @Test
2  fun testCreateUser() {
3      val newUserDto = CreateUserDto(
4          username = "New User"
5      )
6      every { userRepository.save(any()) } returns User(
7          username = "New User"
8      )
9      val actual = userService.createUser(newUserDto)
10     assertEquals(newUserDto.username, actual.username)
11 }
```

Kotlin

(1)

- save Funktion im Repository wird definiert
- Erstellung eines neuen Users mit passendem Username

Mockk Beispiel

```
1  @Test
2  fun testCreateUser() {
3      val newUserDto = CreateUserDto(
4          username = "New User"
5      )
6      every { userRepository.save(any()) } returns User(
7          username = "New User"
8      )
9      val actual = userService.createUser(newUserDto) } (1)
10     assertEquals(newUserDto.username, actual.username)
11 }
```



- Aufruf der createUser Funktion im Service
- save Funktion, die definiert wurde wird genutzt zur User Erstellung

Mockk Beispiel

```
1  @Test
2  fun testCreateUser() {
3      val newUserDto = CreateUserDto(
4          username = "New User"
5      )
6      every { userRepository.save(any()) } returns User(
7          username = "New User"
8      )
9      val actual = userService.createUser(newUserDto)
10     assertEquals(newUserDto.username, actual.username) } (1)
11 }
```



- Abgleichen vom gewollten Username und dem Username vom erstellen User
- Wenn beide Namen gleich sind, war der Test erfolgreich

Integrations-Tests

Integrations-Tests

- Tests, die mehrere Bestandteile des Gesamtsystems testen
- Bestandteile müssen in Tests zusammen funktionieren
- Unterschiedliche Module eines Systems sollen sich so verhalten wie erwartet
- Kommen meist nach Unit Tests und vor End-To-End Tests

Beispiel Integrations-Tests in Spring:

- Testing von allen API Endpunkten auf Funktionalität

Integrations-Tests

Integrations-Test Beispiel

Integrations-Test Beispiel

```
1  @SpringBootTest
2  @AutoConfigureMockMvc
3  class UserIntegrationTest {
4      @Autowired
5      lateinit var mockMvc: MockMvc
6      val mapper = jacksonObjectMapper()
7
8      /*
9      Test Methoden
10     */
11
12 }
```

 Kotlin

Integrations-Tests

GET Request Test

Integrations-Test Beispiel

GET Request Test

```
1 @Test
2 fun `get users returns success`() {
3     mockMvc.get("/users")
4         .andDo { MockMvcResultHandlers.print() }
5         .andExpect { status { isOk() } }
6 }
```

 Kotlin

Integrations-Test Beispiel

GET Request Test

```
1 @Test
2 fun `get users returns success`() {
3     mockMvc.get("/users")
4         .andDo { MockMvcResultHandlers.print() }
5         .andExpect { status { isOk() } }
6 }
```

◀ Kotlin

```
1 @RestController
2 @RequestMapping("/users")
3 class UserController
```

◀ Kotlin

- GET Funktion wird mit der /users URL aufgerufen

Integrations-Test Beispiel

GET Request Test

```
1 @Test
2 fun `get users returns success`() {
3     mockMvc.get("/users")
4         .andDo { MockMvcResultHandlers.print() }
5         .andExpect { status { isOk() } }
6 }
```

 Kotlin

- Ausgabe der Ergebnisse des MockMVC nach der Anfrage (Optional)

Integrations-Test Beispiel

GET Request Test

```
1 @Test
2 fun `get users returns success`() {
3     mockMvc.get("/users")
4         .andDo { MockMvcResultHandlers.print() }
5         .andExpect { status { isOk() } }
6 }
```

◀ Kotlin

- Überprüfung der Request
- Status Code wird abgeglichen
- `.isOk()` überprüft auf Code 200

Integrations-Tests

POST Request Test

Integrations-Test Beispiel

POST Request Test

```
1  @Test
2  fun `post users returns success`() {
3      val newUser = CreateUserDto(username = "New User")
4
5      mockMvc.post("/users") {
6          contentType = MediaType.APPLICATION_JSON
7          content = jsonMapper().writeValueAsString(newUser)
8          accept = MediaType.APPLICATION_JSON
9      }.andExpect {
10         status { isCreated() }
11     }
12 }
```

 Kotlin

Integrations-Test Beispiel

POST Request Test

```
1 val newUser = CreateUserDto(username = "New User")
```

 Kotlin

- POST Request benötigt ein CreateUserDto
- DTO wird mit einem Username erstellt

Integrations-Test Beispiel

POST Request Test

```
1 mockMvc.post("/users") {  
2     contentType = MediaType.APPLICATION_JSON  
3     accept = MediaType.APPLICATION_JSON  
4     content = jsonMapper().writeValueAsString(newUser)  
5 }.andExpect {  
6     status { isCreated() }  
7 }
```

◀ Kotlin

- .post Funktion wird mit der passenden URL als Parameter aufgerufen
- Content Typen werden festgelegt
- DTO wird mit einem JSON Mapper zu einem String umgewandelt
- .andExpect überprüft nach der Request ob die angegebenen Bedingungen erfüllt wurden

Integrations-Test Beispiel

POST Request Test

```
1 mockMvc.post("/users") {  
2     contentType = MediaType.APPLICATION_JSON  
3     accept = MediaType.APPLICATION_JSON  
4     content = jsonMapper().writeValueAsString(newUser)  
5 }.andExpect {  
6     status { isCreated() }  
7 }
```

◀ Kotlin

- .post Funktion wird mit der passenden URL als Parameter aufgerufen

Integrations-Test Beispiel

POST Request Test

```
1 mockMvc.post("/users") {  
2     contentType = MediaType.APPLICATION_JSON  
3     accept = MediaType.APPLICATION_JSON  
4     content = jsonMapper().writeValueAsString(newUser)  
5 }.andExpect {  
6     status { isCreated() }  
7 }
```

◀ Kotlin

- .post Funktion wird mit der passenden URL als Parameter aufgerufen
- Content Typen werden festgelegt

Integrations-Test Beispiel

POST Request Test

```
1 mockMvc.post("/users") {  
2     contentType = MediaType.APPLICATION_JSON  
3     accept = MediaType.APPLICATION_JSON  
4     content = jsonMapper().writeValueAsString(newUser)  
5 }.andExpect {  
6     status { isCreated() }  
7 }
```

Kotlin

- .post Funktion wird mit der passenden URL als Parameter aufgerufen
- Content Typen werden festgelegt
- DTO wird mit einem JSON Mapper zu einem String umgewandelt

Integrations-Test Beispiel

POST Request Test

```
1 mockMvc.post("/users") {  
2     contentType = MediaType.APPLICATION_JSON  
3     accept = MediaType.APPLICATION_JSON  
4     content = jsonMapper().writeValueAsString(newUser)  
5 } .andExpect {  
6     status { isCreated() }  
7 }
```

◀ Kotlin

- .post Funktion wird mit der passenden URL als Parameter aufgerufen
- Content Typen werden festgelegt
- DTO wird mit einem JSON Mapper zu einem String umgewandelt
- .andExpect überprüft nach der Request ob die angegebenen Bedingungen erfüllt wurden

Integrations-Tests

Erweiterter POST Request Test

Integrations-Test Beispiel

Erweiterter POST Request Test

```
1  @Test
2  fun `post todoItems returns success`() {
3      val newUserGetDto: GetUserDto = createNewUser()
4      val newTodoItem = CreateTodoItemDto(/* Values */)
5
6      mockMvc.post("/todos") {
7          contentType = MediaType.APPLICATION_JSON
8          content = mapper.writeValueAsString(newTodoItem)
9          accept = MediaType.APPLICATION_JSON
10     }.andExpect {
11         status { isCreated() }
12     }
13 }
```

◀ Kotlin

Integrations-Test Beispiel

Erweiterter POST Request Test

Integrations Test für ein Todo Item. Todo Item braucht einen User. Der Test muss diesen User erstellen.

Test-Schritte:

1. Erstellung eines Users mit `mockMvc.post("/users")`
2. Erstellung eines `CreateTodoItemDto` mit der User ID
3. Ausführen von `mockMvc.post("/todos")` mit dem DTO als JSON String im Content
4. Überprüfung des Status Codes der POST Request

Integrations-Test Beispiel

Erweiterter POST Request Test

1. Erstellung eines Users mit `mockMvc.post("/users")`

```
1  fun createUser(): GetUserDto {
2      val newUser = CreateUserDto(username = "New User")
3      val postUserResult = mockMvc.post("/users") {
4          contentType = MediaType.APPLICATION_JSON
5          content = mapper.writeValueAsString(newUser)
6          accept = MediaType.APPLICATION_JSON
7      }.andExpect {
8          status { isCreated() }
9      }.andReturn()
10     val newUserAsString = postUserResult.response.contentAsString
11     return mapper.readValue<GetUserDto>(newUserAsString)
12 }
```

◀ Kotlin

Integrations-Test Beispiel

Erweiterter POST Request Test

1. Erstellung eines Users mit `mockMvc.post("/users")`

```
1 val postUserResult = mockMvc.post("/users") {  
2     contentType = MediaType.APPLICATION_JSON  
3     content = mapper.writeValueAsString(newUser)  
4     accept = MediaType.APPLICATION_JSON  
5 }.andExpect {  
6     status { isCreated() }  
7 }.andReturn()
```

◀ Kotlin

- `.andReturn()` gibt das Ergebnis der Request als Result aus
- Content vom Result kann als String ausgelesen werden

Integrations-Test Beispiel

Erweiterter POST Request Test

1. Erstellung eines Users mit `mockMvc.post("/users")`

```
1 val newUserAsString = postUserResult
2   .response
3   .contentAsString
```

◀ Kotlin

- Mit `.response.contentAsString` kann der Content als JSON String erhalten werden
- JSON String kann mit JSON Mapper zu einem `GetUserDto` umgewandelt werden um die ID auszulesen

```
1 mapper.readValue<GetUserDto>(newUserAsString)
```

◀ Kotlin

Integrations-Test Beispiel

Erweiterter POST Request Test

2. Erstellung eines CreateTodoItemDto mit der User ID

```
1 val newTodoItem = CreateTodoItemDto(  
2     name = "Test Todo Name",  
3     description = "Test Todo Description",  
4     done = false,  
5     created = Date(),  
6     shouldBeDoneBy = Date(),  
7     userId = newUserGetDto.id  
8 )
```

◀ Kotlin

Integrations-Test Beispiel

Erweiterter POST Request Test

3. Ausführen von `mockMvc.post("/todos")` mit dem DTO als JSON String im Content

```
1 mockMvc.post("/todos") {  
2     contentType = MediaType.APPLICATION_JSON  
3     content = mapper.writeValueAsString(newTodoItem)  
4     accept = MediaType.APPLICATION_JSON  
5 }
```

◀ Kotlin

Integrations-Test Beispiel

Erweiterter POST Request Test

4. Überprüfung des Status Codes der POST Request

```
1 .andExpect {  
2     status { isCreated() }  
3 }
```

 Kotlin

Projektbeispiele

Einfache Finanzapp

Backend:

- Entities: Konto, Transaktionen, Benutzer
- Benutzer
 - Konto 1
 - Transaktionen 1
 - Transaktionen 2
 - ...
 - Konto 2
 - Transaktionen 1
 - Transaktionen 2
 - ...
 - ...

Frontend:

- CRUD für alle Entities
- Übersicht über Finanzen

Notensoftware

Backend:

- Entities: Student, Modul
- Student
 - Modul 1
 - Modul 2
 - ...
- Berechnung von Notendurchschnitten
- ECTS Berechnung

Frontend:

- CRUD für alle Entities
- Übersicht über Noten und Durchschnitte

Einbindung von externen APIs

- Flugdaten sammeln mit der Lufthansa API mit Suche nach unterschiedlichen Parametern in Spring/Frontend
 - Wetterdaten sammeln mit Suche nach unterschiedlichen Parametern in Spring/Frontend
 - DB API
- ...

MENSA IST VORBEI